3

Introdução a classes e objetos

OBJETIVOS

Neste capítulo você aprenderá:

- O que são classes, objetos, métodos e variáveis de instância.
- Como declarar uma classe e utilizá-la para criar um objeto.
- Como declarar métodos em uma classe para implementar os comportamentos da classe.
- Como declarar variáveis de instância em uma classe para implementar os atributos da classe.
- Como chamar o método de um objeto para fazer esse método realizar sua tarefa.
- As diferenças entre variáveis de instância de uma classe e variáveis locais de um método.
- Como utilizar um construtor para assegurar que os dados de um objeto sejam inicializados quando o objeto for criado.
- As diferenças entre tipos primitivos e referências.

Outline

3.1 Introdução

3.2 Classes, Objetos, Métodos e Variáveis de Instância

3.3 Declarando uma classe com um método e instanciando um objeto de uma classe

3.4 Declarando um método com um parâmetro

3.5 Variáveis de Instância, métodos *set* e *get*

3.6 Tipos primitivos vs. tipos por referência

3.7 Inicializando objetos com construtores

3.8 Números de ponto flutuante e o tipo `double`

3.9 (Optional) GUI and Graphics Case Study: Using Dialog Boxes

3.10 (Optional) Software Engineering Case Study: Identifying the Classes in a Requirements Document

3.11 Conclusão

3.1 Introdução

- **Classes**
- **Números de ponto flutuante**

3.2 Classes, Objetos, Métodos e Variáveis de Instância

- **Classe provê um ou mais métodos**
- **Método representa tarefa em um programa**
  - **Descreve mecanismos que realmente realizam suas tarefas**
  - **Oculta do seu usuário as tarefas complexas que ele realiza**
  - **Chamada ao método instrui o método a realizar sua tarefa**

3.2 Classes, Objetos, Métodos e Variáveis de Instância (Cont.)

- **Classes contém um ou mais atributos**
  - **Especificados pelas variáveis de instância**
  - **Desenvolvidas com o objeto e seu uso**

3.3 Declarando uma classe com um método e instanciando um objeto de uma classe

- **Cada declaração de classe que inicia com a plavra-chave `public` deve ser armazenada em um arquivo que tenha o mesmo nome da classe e terminar com a extensão de nome do arquivo `.java`.**

Classe `GradeBook`

- **Palavra-chave `public` é um modificador de acesso**
- **Declarações de classe incluem:**
  - **Modificador de acesso**
  - **Palavra-chave `class`**
  - **Par de chaves esquerda e direita**

Classe `GradeBook`

- **Declarações de método**
  - **Palavra-chave `public` indica método disponível "ao público"**
  - **Palavra-chave `void` indica tipo sem retorno**
  - **Modificador de acesso, tipo de retorno, nome de método e parênteses compreendem o cabeçalho do método**

Erro comum de programação 3.1

Declarar mais de uma classe `public` no mesmo arquivo é um erro de compilação.

```java
// Fig. 3.1: GradeBook.java
// Declaração de classe com um método.

public class GradeBook
{
   // Exibe uma mensagem de boas-vindas para o usuário GradeBook
   public void displayMessage()
   {
      System.out.println( "Bem-vindo a Grade Book!" );
   } // fim do método displayMessage()

} // fim classe GradeBook
```

Imprime uma linha de texto na saída (tela)

Classe `GradeBookTest`

- **Java é extensível**
  - **Programadores podem criar novas classes**
- **Expressão de criação de instância de classe**
  - **Palavra-chave `new`**
  - **E o nome da classe para criar mais os parênteses**
- **Chamando um método**
  - **Nome do objeto, então um ponto separador (`.`)**
  - **Então, nome do método e parênteses**

```
1  // Fig. 3.2: GradeBookTest.java
2  // Cria um objeto GradeBook e chama seu método displayMessage.
3
4  public class GradeBookTest
5  {
6     // Método main inicia a execução do programa
7     public static void main( String args[] )
8     {
9        // Cria um objeto GradeBook e o a
10       GradeBook meuGradeBook = new Grad
11
12       // Chama o método displayMessage
13       meuGradeBook.displayMessage();
14    } // fim do main
15
16 } // fim da classe GradeBookTest
```

Usa expressão de criação de instância de classe para criar objeto da classe `GradeBook`

Chama o método `displayMessage` usando objeto `GradeBook`

```
Bem-vindos a Grade Book!
```

Compilando um aplicativo com múltiplas Classes

- **Compilando múltiplas classes**
  - **Liste cada arquivo `.java` separados por espaço**
  - **Compile com `*.java` para compilar todos os arquivos `.java` no diretório**

Diagrama de classe UML para a classe GradeBook

- **Diagramas de classe UML**
  - **Compartimento superior contém o nome da classe**
  - **O compartimento do meio contém atributos da classe ou variáveis de instância**
  - **O compartimento inferior contém as operações da classe ou métodos**
    - **Sinal + indica método** `public`

Fig. 3.3 | Diagrama da classe em UML indicando que a classe `GradeBook` tem uma operação `public displayMessage`.

3.4 Declarando um método com um parâmetro

- **Parâmetros de método**
  - **Informação adicional passada para um método**
  - **Fornecidos para chamar um método com argumentos**

3.4 Declarando um método com um parâmetro

- **Métodos em** `Scanner`
  - `nextLine` **lê a próxima linha de entrada**
  - `next` **lê a próxima palavra de entrada**

```java
// Fig. 3.4: GradeBook.java
// Declaração de classe com um método que possui um parâmetro.

public class GradeBook
{
   // Exibe uma mensagem de boas-vindas para o usuário GradeBook
   public void displayMessage( String nomeDoCurso )
   {
      System.out.printf( "Bem-vindos a disciplina\n%s!\n",
         nomeDoCurso );
   } // fim do método displayMessage

} // fim da classe GradeBook
```

Chama o método `printf` com `nomeDoCurso` como argumento

```java
// Fig. 3.5: GradeBookTest.java
// Cria objeto GradeBook e passa uma String para
// seu método displayMessage.
import java.util.Scanner; // programa usa Scanner

public class GradeBookTest
{
   // método main inicia a execução do programa
   public static void main( String args[] )
   {
      // Cria Scanner para obter entrada a partir da janela de comando
      Scanner entrada = new Scanner( System.in );

      // Cria um objeto GradeBook e o atribui a meuGradeBook
      GradeBook meuGradeBook = new GradeBook(

      // prompt para entrada do nome do curso
      System.out.println( "Digite o nome do curso:" );
      String disciplina = entrada.nextLine(); // lê uma linha de texto
      System.out.println(); // linha em branc

      // chama método displayMessage de meuGr
      // e passa nameOfCourse como um argumento
      meuGradeBook.displayMessage( disciplina );
   } // fim do main

} // fim da classe GradeBookTest
```

Chama o método `nextLine` para ler uma linha de entrada

Chama `displayMessage` com um argumento

```
Digite o nome do curso:
Algoritmos e Linguagem de Programação II

Bem-vindos a disciplina
Algoritmos e Linguagem de Programação II!
```

Observação de engenharia de software 3.1

Normalmente os objetos são criados com `new`. Uma exceção é um literal de string que está entre aspas, como `"hello"`. Os literais de string são referências a objetos `String` que são criados implicitamente pelo Java.

Mais sobre Argumentos e Parâmetros

- **Parâmetros são especificados na lista de parâmetros do método**
  - **Parte do cabeçalho do método**
  - **Use vírgula para separar os parâmetros**

Erro comum de programação 3.2

Ocorrerá um erro de compilação se o número de argumentos em uma chamada de método não corresponder ao número de parâmetros na declaração de método.

Erro comum de programação 3.3

Ocorrerá um erro de compilação se os tipos dos argumentos em uma chamada de método não forem consistentes com os tipos dos parâmetros correspondentes na declaração do método.

Diagrama de classe UML atualizado para a classe `GradeBook`

- **Diagrama de classe UML**
  - **Parâmetros especificados pelo nome do parâmetro seguido de dois-pontos e pelo tipo de parâmetro**

Fig. 3.6 | Diagrama da classe UML indicando que a classe GradeBook tem uma operação displayMessage com um parâmetro courseName de tipo UML String.

Notes sobre declarações `Import`

- `java.lang` **é implicitamente importado em todo programa**
- **Pacote-padrão**
  - **Contém classes compiladas no mesmo diretório**
  - **Implicitamente importado no mesmo arquivo de código-fonte de outras classes no mesmo pacote**
- **Pacotes são desnecessários se nomes completamente qualificados forem usados**

Observação de engenharia de software3.2

O compilador Java não exigirá as declarações `import` em um arquivo de código-fonte Java se o nome de classe completamente qualificado for especificado toda vez que um nome da classe for utilizado no código-fonte. Mas a maioria dos programadores em Java considera incômoda a utilização dos nomes completamente qualificados e, em vez disso, prefere usar declarações `import`.

3.5 Variáveis de instância, métodos *set* e métodos *get*

- **Variáveis declaradas no corpo de um método**
  - **Chamadas variáveis locais**
  - **Só podem ser utilizadas nesse método**
- **Variáveis declaradas numa declaração de classe**
  - **Chamadas de campos ou variáveis de instância**
  - **Cada objeto da classe tem uma instância separada da variável**

```java
// Fig. 3.7: GradeBook.java
// Classe GradeBook que contém uma variável de instância nomeDoCurso
// e métodos para configurar e obter seu valor.

public class GradeBook
{
   private String nomeDoCurso; // nome do curso para esse GradeBook

   // método para configurar o nome do curso
   public void setNomeDoCurso( String nome )
   {
      nomeDoCurso = nome; // armazena o nome do curso
   } // fim do método setNomeDoCurso

   // método para recuperar o nome do curso
   public String getNomeDoCurso()
   {
      return nomeDoCurso;
   } // fim do método getNomeDoCurso

   // exibe uma mensagem de boas-vindas para o usuário GradeBook
   public void displayMessage()
   {
      // essa instrução chama getNomeDoCurso para obter o
      // nome do curso que esse GradeBook representa
      System.out.printf( "Bem-vindo ao GradeBook para\n%s!\n",
         getNomeDoCurso() );
   } // fim do método displayMessage

} // fim da classe GradeBook
```

Variável de instância `nomeDoCurso`

Método *set* para `nomeDoCurso`

Método *get* para `nomeDoCurso`

Chama o método *get*

Modificadores de acesso `public` e `private`

- **Palavra-chave private**
  - **Usada para a maioria das variáveis de instância**
  - **Variáveis `private` e métodos são acessíveis somente a métodos da classe em que são declaradas**
  - **Declarações de variáveis de instância `private` são conhecidas como ocultamento de dados**
- **Tipo de retorno**
  - **Indica o tipo retornado pelo método**
  - **Declarado no cabeçalho do método**

Observação de engenharia de software 3.3

Anteceda cada campo e cada declaração de método com um modificador de acesso. Como regra geral, as variáveis de instância devem ser declaradas `private` e os métodos ser declarados `public`. (Veremos que é apropriado declarar certos métodos `private` se eles forem acessados apenas por outros métodos da classe).

Boa prática de programação 3.1

Preferimos listar campos de uma classe primeiro para que, ao ler o código, você veja os nomes e os tipos das variáveis antes de colocá-los em uso nos métodos da classe. É possível listar os campos da classe em qualquer lugar na classe fora de suas declarações de método, mas sua dispersão tende a resultar em código de difícil leitura.

Boa prática de programação 3.2

Coloque uma linha em branco entre as declarações de método para separar os métodos e aprimorar a legibilidade do programa.

Classe `GradeBookTest` que demonstra a classe `GradeBook`

- **Valor inicial padrão**
  - **Fornecido para todos os campos que não foram inicializados**
  - **Valor `null` para `String`s**

Métodos *set* e *get*

- **Variáveis de instância `private`**
  - **Não podem ser acessadas diretamente pelos clientes do objeto**
  - **Use métodos *set* para alterar valor**
  - **Use méotodos *get* para recuperar valor**

```java
// Fig. 3.8: GradeBookTest.java
// Cria e manipula um objeto GradeBook.
import java.util.Scanner; // programa utiliza Scanner

public class GradeBookTest
{
   // método main inicia a execução do programa
   public static void main( String args[] )
   {
      // cria Scanner para obter entrada a partir da janela de comando
      Scanner entrada = new Scanner( System.in );

      // cria um objeto GradeBook e o atribui a meuGradeBook
      GradeBook meuGradeBook = new GradeBook();

      // exibe valor inicial de nomeDoCurso
      System.out.printf( "Nome inicial do curso é: %s\n\n",
         meuGradeBook.getNomeDoCurso() );

```

Chama o método *get* para `nomeDoCurso`

```
20        // solicita e lê o nome do curso
21        System.out.println( "Digite o nome do curso:" );
22        String oNome = entrada.nextLine(); // lê uma linha de texto
23        meuGradeBook.setNomeDoCurso( oNome ); // configura o nome do curso
24        System.out.println(); // uma linha
25
26        // exibe a mensagem depois de especificar o nome do curso
27        meuGradeBook.displayMessage();
28     } // fim do main
29
30  } // fim da classe GradeBookTest
```

Chama método *set* para `nomeDoCurso`

Chama `displayMessage`

```
Nome inicial do curso é: null

Digite o nome do curso:
Algoritmos e Linguagem de Programação II

Bem-vindo ao GradeBook para
Algoritmos e Linguagem de Programação II!
```

Diagrama de classe UML do `GradeBook` com uma variável de instância e métodos *set* e *get*

- **Atributos**
  - **Listados no compartimento do meio**
  - **Nome do atributo seguido por dois-pontos e tipo**
- **Tipo de retorno de um método**
  - **Indicado com dois-pontos e tipo de retorno depois dos parênteses e do nome da operação**

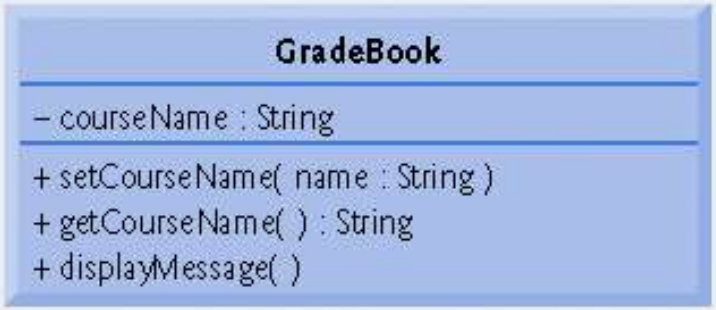


Fig. 3.9 | Diagrama de classe UML indicando que a classe `GradeBook` tem um atributo `courseName` de tipo UML `String` e três operações—`setCourseName` (com um parâmetro de nome de tipo UML `String`), `getCourseName` (retorna o tipo UML `String`) e `displayMessage`.

Tipos primitivos vs. tipos por referência

- **Tipos em Java**
  - **Primitivos**
    - `boolean`, `byte`, `char`, `short`, `int`, `long`, `float`, `double`
  - **Referência (as vezes chamados tipos não-primitivos)**
    - **Objetos**
    - **Valor default** `null`
    - **Usados para invocar (chamar) métodos do objeto**

Observação de engenharia de software 3.4

O tipo declarado de uma variável (por exemplo, `int`, `double` ou `GradeBook`) indica se a variável é de tipo primitivo ou de tipo por referência. Se o tipo de uma variável não for um dos oito tipos primitivos, então ele é um tipo por referência. (Por exemplo, `Conta conta1` indica que `conta1` é uma referência a um objeto `Conta`.)

3.7 Inicializando objetos com construtores

- **Construtor**
  - **Inicializa um objeto de uma classe**
  - **Java necessita de um construtor para cada classe**
  - **Java provê um construtor padrão sem argumento se nenhum for criado**
  - **Chamado quando a palavra-chave** `new` **é seguida pelo nome da classe e parênteses**

```java
// Fig. 3.10: GradeBook.java
// Classe GradeBook com um construtor para inicializar o nome de um curso.

public class GradeBook
{
   private String nomeDoCurso; // nome do curso para esse GradeBook

   // construtor inicializa nomeDoCurso com String fornecida como argumento
   public GradeBook( String nome )
   {
      nomeDoCurso = nome; // inicializa no
   } // fim construtor

   // método para configurar o nome do curso
   public void setNomeDoCurso( String nome )
   {
      nomeDoCurso = nome; // armazena o nome do curso
   } // fim do método setNomeDoCurso

   // método para recuperar o nome do curso
   public String getNomeDoCurso()
   {
      return nomeDoCurso;
   } // fim do método getNomeDoCurso
```

Construtor para inicializar a variável `nomeDoCurso`

```
25
26    // exibe uma mensagem de boas-vindas para o usuário GradeBook
27    public void displayMessage()
28    {
29       // essa instrução chama getNomeDoCurso para obter o
30       // nome do curso que esse GradeBook representa
31       System.out.printf( "Bem-vindo ao grade book para\n%s!\n",
32          getNomeDoCurso() );
33    } // fim do método displayMessage
34
35 } // fim da classe GradeBook
```

```java
// Fig. 3.11: GradeBookTest.java
// Construtor GradeBook utilizado para especificar o nome do curso na
// hora em que cada objeto GradeBook é criado.

public class GradeBookTest
{
   // método main inicia a execução do programa
   public static void main( String args[] )
   {
      // cria objeto GradeBook
      GradeBook gradeBook1 = new GradeBook(
         "Algoritmos e Linguagem de Programação II" );
      GradeBook gradeBook2 = new GradeBook(
         "Engenharia de Software" );

      // exibe valor inicial de nomeDoCurso para cada GradeBook
      System.out.printf( "Nome do curso no gradeBook1 é: %s\n",
         gradeBook1.getNomeDoCurso() );
      System.out.printf( "Nome do curso no gradeBook2 é: %s\n",
         gradeBook2.getNomeDoCurso() );
   } // fim do main

} // fim da classe GradeBookTest
```

Chama o construtor para criar primeiro objeto GradeBook

Cria segundo objeto GradeBook

```
Nome do curso no gradeBook1 é: Algoritmos e Linguagem de Programação II
Nome do curso no gradeBook2 é: Engenharia de Software
```

Dica de prevenção de erros 3.1

A menos que a inicialização-padrão de variáveis de instância de sua classe seja aceitável, forneça um construtor para assegurar que as variáveis de instância de sua classe sejam adequadamente inicializadas com valores significativos quando cada novo objeto de sua classe for criado.

Adicionando o construtor ao diagrama de classe UML da classe `GradeBook`

- **Diagrama de classe UML**
  - **Construtores ficam no terceiro compartimento**
  - **Localização: “<<constructor>>” antes do nome do construtor**
  - **Por convenção, coloque os construtores antes de outras operações no seu compartimento**

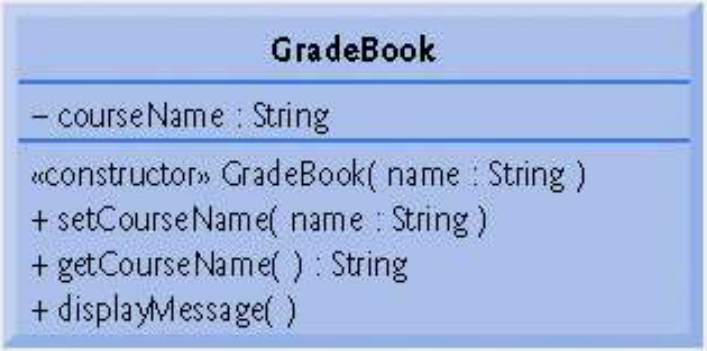


Fig. 3.12 | Diagrama de classe UML que indica que a classe `GradeBook` tem um construtor com um parâmetro `name` de tipo UML `String`.

3.8 Números de ponto flutuante e tipo `double`

- **Números de ponto flutuante**
  - `float`
  - `double`
    - **Armazena números com mais magnitude e precisão que o** `float`

Precisão de número de ponto flutuante e requisitos de memória

- **float**
  - **Números de ponto flutuante de precisão simples**
  - **Sete dígitos significativos**
- **double**
  - **Números de ponto flutuante de dupla precisão**
  - **Quinze dígitos significativos**

Erro comum de programação 3.4

Usar números de ponto flutuante de uma maneira que assume que eles são representados precisamente pode levar a erros de lógica.

```java
// Fig. 3.13: Account.java
// Classe Account com um construtor para
// inicializar a variável de instância saldo.

public class Account
{
   private double saldo; // variável de instância que armazena o saldo

   // construtor
   public Account( double saldoInicial )
   {
      // validação para que saldoInicial seja maior que 0.0;
      // se não, o saldo é inicializado com o valor padrão 0.0
      if ( saldoInicial > 0.0 )
         saldo = saldoInicial;
   } // fim do construtor Account

   // credita (adiciona) uma quantia a conta
   public void credit( double quantia )
   {
      saldo = saldo + quantia; // adiciona quantia ao saldo
   } // fim do método credit

   // retorna o saldo da conta
   public double getSaldo()
   {
      return saldo; // fornece o valor de saldo ao método chamador
   } // fim do método getSaldo

} // fim da classe Account
```

Variável double saldo

Classe `AccountTest` para usar `Account`

- **Especificador de formato `%f`**
  - **Utilizado para saída com números em ponto flutuante**
  - **Um valor decimal entre a porcentagem e `f` altera a precisão do número**

```java
// Fig. 3.14: AccountTest.java
// Cria e manipula um objeto Account.
import java.util.Scanner;

public class AccountTest
{
   // método principal incia a execução do aplicativo Java
   public static void main( String args[] )
   {
      Account conta1 = new Account( 50.00 ); // cria objeto Account
      Account conta2 = new Account( -7.53 ); // cria objeto Account

      // display initial balance of each object
      System.out.printf( "Saldo da conta1: $%.2f\n",
         conta1.getSaldo() );
      System.out.printf( "Saldo da conta2: $%.2f\n\n",
         conta2.getSaldo() );

```

```
19        // cria Scanner para obter entrada a partir da janela de comando
20        Scanner input = new Scanner( System.in );
21        double quantiaDeposito; // quantia de depósito lida do usuário
22
23        System.out.print( "Entre com o valor para a conta1: " ); // prompt
24        quantiaDeposito = input.nextDouble(); // obtém a entrada do usuário
25        System.out.printf( "\nadicionando %.2f ao saldo da conta1\n\n",
26           quantiaDeposito );
27        conta1.credit( quantiaDeposito ); // adiciona ao saldo de conta1
28
29        // exibe saldos
30        System.out.printf( "saldo da conta1: $%.2f\n",
31           conta1.getSaldo() );
32        System.out.printf( "saldo da conta2: $%.2f\n\n",
33           conta2.getSaldo() );
34
35        System.out.print( "Enter com o valor para a conta2: " ); // prompt
36        quantiaDeposito = input.nextDouble(); // obtém entrda do usuário
37        System.out.printf( "\nadicionando %.2f ao saldo da conta2\n\n",
38           quantiaDeposito );
39        conta2.credit( quantiaDeposito ); // adiciona ao saldo de conta2
```

Entre com um valor double

Entre com um valor double

```
41        // exibe os saldos
42        System.out.printf( "saldo da conta1: $%.2f\n",
43           conta1.getSaldo() );
44        System.out.printf( "saldo da conta2: $%.2f\n",
45           conta2.getSaldo() );
46     } // fim do main
47
48 } // fim da classe AccountTest
```

Saída: um valor `double`

```
saldo da conta1: $50.00
saldo da conta2: $0.00

Entre com o valor para a conta1: 25.53

adicionando 25.53 ao saldo da conta1

saldo da conta1: $75.53
saldo da conta2: $0.00

Entre com o valr para a conta2: 123.45

adicionando 123.45 ao saldo da conta2

saldo da conta1: $75.53
saldo da conta2: $123.45
```

Fig. 3.15 | Diagrama de classe UML para indicar que a classe `Account` possui um atributo `private balance` do tipo UML `Double`, um construtor (com um parâmetro do tipo UML `Double`) e duas operações `public–credit` (com um parâmetro `amount` do tipo UML `Double`) e `getBalance` (retorna o tipo UML `Double`).

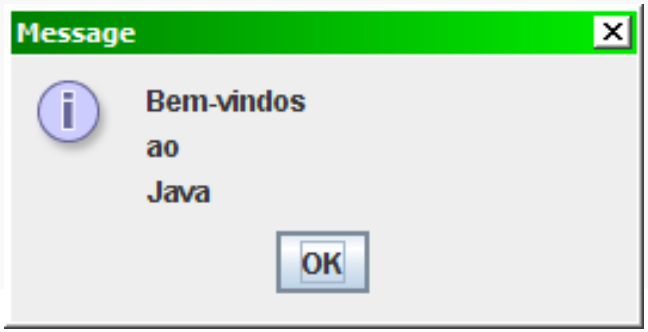
Exibindo texto numa caixa de diálogo

- **Janelas e caixas de diálogo**
  - **A maioria das aplicações Java as usam para exibir sua saída**
  - `JOptionPane` **provê caixas de diálogo ao usuário**

```
1  // Fig. 3.17: Dialog1.java
2  // Exibindo múltiplas linhas na caixa de diálogo.
3  import javax.swing.JOptionPane; // importa a classe JOptionPane
4
5  public class Dialog1
6  {
7     public static void main( String arg
8     {
9        // exibe um diálogo com a mensagem
10       JOptionPane.showMessageDialog( null, "Bem-vindos\nao\nJava" );
11    } // fim do main
12 } // fim da classe Dialog1
```

Importa aclasse `JOptionPane`

Exibe a mensagem numa caixa de diálogo



Exibindo texto numa caixa de diálogo

- **Pacote** `javax.swing`
  - **Contém classes para ajudar a criar interfaces gráficas com usuário (GUIs)**
  - **Uma delas é a classe** `JOptionPane`
    - **Declara o método** `static` `showMessageDialog` **pra exibir uma mensagem na caixa de diálogo**

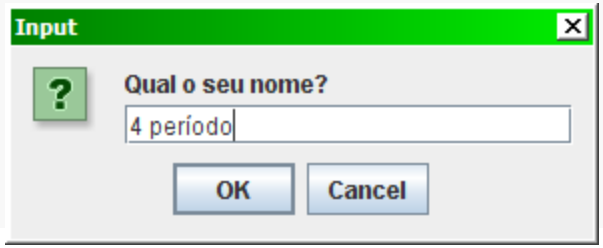
Inserindo texto numa caixa de diálogo

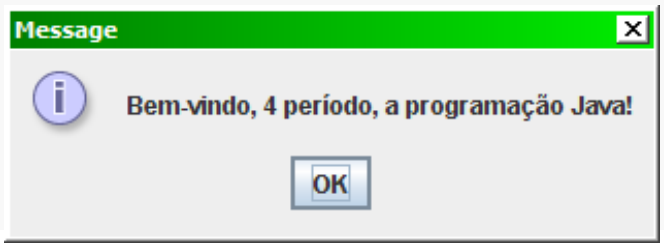
- **Input dialog**
  - **Permite ao usuário entrar com informações**
  - **Criado usando o método** `showInputDialog` **da classe** `JOptionPane`

```java
// Fig. 3.18: NameDialog.java
// Entrada básica com caixa de diálogo.
import javax.swing.JOptionPane;

public class NameDialog
{
   public static void main( String args[] )
   {
      // pede ao usuário para digitar seu nome
      String nome =
         JOptionPane.showInputDialog( "Qual o seu nome?" );

      // cria a mensagem
      String message =
         String.format( "Bem-vindo, %s, a programação Java!", nome );

      // exibe a mensagem para cumprimentar o usuário
      JOptionPane.showMessageDialog( null, message );
   } // fim do main
} // fim da classe NameDialog
```

Show input dialog

Formatação de `String` para saída do usuáriio





Ícones em JOptionPane

Tipo de diálogo	Ícone	Descrição
`JOptionPane.ERROR_MESSAGE`		Indica erro.
`JOptionPane.INFORMATION_MESSAGE`		Indica uma mensagem de informação.
`JOptionPane.WARNING_MESSAGE`		Indica um diálogo com problema em potencial.
`JOptionPane.QUESTION_MESSAGE`		Indica uma pergunta/questionamento.
`JOptionPane.PLAIN_MESSAGE`	Sem ícone	Mensagem simples, sem ícone.

`Constantes JOptionPane` para diálogos.

Referências

- **Adaptado de: “Teaching Resources for Java How to Program, 6/e”.**
- **Deitel & Deitel. Java como programar, 6 ed.**